Zenith in MyKey: integration guide

For: the Claude Code session building the MyKey keyboard. Covers steps 6 and 7 of ZENITH_PLAN.pdf. Steps 1 to 5 (corpus, tokenizer, training, eval, export) are done; this document says what was built, the numbers it reached, the exact model contract, and how to wire it into MyKey.

Repo: **github.com/ItzAditya43/Assignment** (branch main, commit 9df481c). Everything MyKey needs is in **export/**.

1. What you receive

File	Size	Purpose
export/zenith_kb.onnx	18.3 MB	The model. int8, ONNX opset 17, KV cache as explicit inputs/outputs.
export/zenith_kb.tok	0.6 MB	Tokenizer for the phone: plain-text Unigram pieces and scores (format in section 4).
export/tokenizer_kb.json	2.1 MB	Same tokenizer in Hugging Face format; use it to generate test vectors.
export/keyboard_eval*.json, parity.json	small	Measured results (section 2).
export/MYKEY_HANDOFF.md	small	Short version of this document.
zenith/evaluation/keyboard_eval.py	code	Working Python reference for word prediction, prefix completion, token healing and kanji scoring. Port its logic.

2. Where the model stands

Model: 17.8M parameters (7 layers, dim 320, 5 heads, 1 KV head, SwiGLU 864, RoPE, tied 32k embedding), 64-token context. Trained 40,000 steps on 1.34B tokens of chat-style text (OpenSubtitles, Tatoeba, Wikipedia, CC-100 ja), mixed 35% English, 30% Spanish, 35% Japanese. Final validation perplexity: en 47.0, es 50.2, ja 117.6.

Keyboard eval on held-out subtitle sentences (150 per language; the same sentences for both columns). "int8" means the shipped ONNX file itself was scored.

Metric	Zenith fp32	Zenith int8 (shipped)	Bigram baseline	Plan target
English next-word top-3	36.7%	36.6%	20.8%	≥ 35% met
English prefix top-3	74.8%	74.7%	62.7%	≥ 70% met
English keystroke savings	53.0%	53.2%	44.9%	n/a
Spanish next-word top-3	34.2%	see note	19.1%	≥ 30% met
Spanish prefix top-3	68.2%	see note	56.1%	≥ 70% just missed
Spanish keystroke savings	51.6%	see note	42.1%	n/a
Japanese kanji choice top-1	81.0%	see note	88.0%	beat Mozc order (59.6%): met
Japanese next-phrase top-3	41.2%	see note	43.8%	n/a

Note: the int8 Spanish and Japanese runs were still going when this was written; English int8 equals fp32 to within 0.2 points, so expect the same. Results will be in export/keyboard_eval_int8.json.

Honest caveats. (1) The plain bigram beats Zenith on Japanese kanji choice and next-phrase. The bigram was built from the same subtitle corpus as the test sentences, which flatters it, but Zenith does not win every Japanese metric. Treat Zenith as a re-ranker that is *combined* with Mozc cost and user history, not a replacement. (2) Parity: int8 vs fp32 top-1 agreement is 94 to 95%, and the fp32 top-1 is in the int8 top-3 99.5% of the time, but the plan's strict "identical top-5 set" is 76 to 80% (target 95%). Keyboard metrics show no measurable loss, so it is fine to ship. (3) Latency of about 2 ms per token was measured on a PC CPU, not the Xiaomi 14 Civi. Measure it first.

3. Model contract (ONNX)

Name	Type / shape	Meaning
tokens	int64 [1, T]	New token ids, T ≥ 1. Feed the whole context once, then 1 token per keystroke.
pos	int64 [1]	Position of tokens[0] = number of tokens already in the cache.
past_k, past_v	float32 [7, 1, 1, 64, 64]	KV cache (layers, batch, kv_heads, 64 slots, head_dim). Start with zeros.
logits	float32 [1, T, 32000]	Next-token logits at every input position. Use the last row for the next token.
new_k, new_v	same as past	Feed back as past_k / past_v on the next call.

- **Language tag first, always.** Contexts are <lang> + text. Ids: <pad>0 <bos>1 <eos>2 <unk>3 <en>4 <es>5 <ja>6, with <ru>7 and <zh>8 reserved. No <bos> is used in training. <eos> separates lines/messages, so a newline in the text field can map to <eos>.
- **64-position window.** When pos + T would pass 64: zero the cache, set pos = 0, and re-feed <lang> + the last ~47 tokens. RoPE positions are absolute, so you cannot slide the cache. Training rows also often start mid-sentence after the tag, so a truncated context is in-distribution.
- **Batch is 1.** To score N candidates, either run them one by one from a shared cache copy (cheap: 1 to 3 tokens each) or call with T = candidate length on a copied cache.
- Runtime: ONNX Runtime Mobile, CPU execution provider. Transformer weights are stored int8 and expanded at session load (about 30 MB of fp32), plus about 10 MB for the int8 head/embedding. The estimated total is around 45 MB, under the 70 MB budget. Verify on the device.

4. Tokenizer on the phone

```
line 1:  ZENITH-UNIGRAM 1
line 2:  {"vocab_size":32000,"unk_id":3,"byte_fallback":true,"normalizer":"NFKC",
          "metaspace":{"replacement":"\u2581","prepend_scheme":"always"},
          "special":{"<pad>":0,...,"<zh>":8},"byte_offset":9}
line 3+: <piece>\t<score>      one line per id, in id order (\t \n \\ escaped)
```

Encode:

- NFKC-normalise (java.text.Normalizer.Form.NFKC). Replace ' ' with U+2581 and prepend one U+2581.
- Viterbi over the string: maximise the summed piece scores. Skip special ids (0 to 8) and the byte pieces (<0x00>..<0xFF> = ids 9 to 264). Max piece length is 16.
- If no piece covers a character, emit its UTF-8 bytes as ids byte_offset + byte (this is byte fallback; never emit <unk>).
- Decode: concatenate pieces, U+2581 becomes a space, drop the leading space, and join runs of <0xNN> back into UTF-8.

Verified: a 40-line standalone Viterbi that reads only this file matched the Hugging Face tokenizer on 1,200 of 1,200 held-out lines (en, es, ja) including emoji. Build a JVM unit test from tokenizer_kb.json outputs to keep the Kotlin port honest.

5. Building engine/ZenithModel.kt

Suggested structure (matches the plan's section 6):

```
class ZenithModel(ctx: Context) {
  // load once, on a background thread
  private val env = OrtEnvironment.getEnvironment()
  private val session = env.createSession(assetBytes("zenith_kb.onnx"),
                        OrtSession.SessionOptions().apply { setIntraOpNumThreads(2) })
  private val tok = UnigramTokenizer.load(asset("zenith_kb.tok"))

  // per text field: cached state
```

```
    private var ids = IntArray(0); private var k: FloatArray; private var v: FloatArray; private var pos = 0

    fun onContextChanged(lang: Lang, textBeforeCursor: String) { ... }   // diff ids, feed only new tokens
    fun nextLogprobs(): FloatArray { ... }                               // log-softmax of last logits row
    fun scoreContinuation(tokenIds: IntArray): Float { ... }             // sum log-probs on a cache copy
}
```

- Run on a single background executor. Every keystroke cancels the previous request (keep a generation counter and drop stale results).
- Keep the cache between keystrokes. On each change, find the longest common token prefix with the cached ids. If the user appended text, feed only the new tokens; if they deleted or edited, roll back by re-feeding from the common prefix (re-prefill is about 64 tokens, which is cheap).
- Fallback: if the model fails to load or a call takes longer than the budget (for example 30 ms), return the current dictionary suggestions unchanged. Zenith only re-orders and adds.

6. English and Spanish: suggestions and autocorrect

Do **not** feed the half-typed word as context. Feed the text up to the last space, then score candidate words.

- **Candidates** come from MyKey's existing dictionary and learned words, filtered by the typed letters (prefix and autocorrect neighbours) as today.
- **Zenith score per candidate w** = log P(tokens of ' w' | context) + log P(word ends), where 'ends' = probability mass of the next token starting with U+2581, being punctuation, or <eos>. Tokenise ' w' with the leading U+2581 (word-start). keyboard_eval.py LM.predict_words does exactly this with a small beam.
- **Mix:** final = a*zenith + b*log(word frequency) + c*user_habit, as the plan says. Start with a=1, b=0.3, c=1, and tune on your own typing logs. Normalise zenith over the candidate set (log-softmax among candidates).
- **Next word (nothing typed yet):** take the top tokens starting with U+2581, extend each greedily or with a beam of up to 3 to 4 tokens until a word-end, and keep the top 3. Measured: 36.6% top-3 in English and 34% in Spanish.
- Autocorrect ("I want to by a car" becomes "buy"): include the typed word and its edit-distance neighbours as candidates, score each with Zenith in context, and only replace when the margin over the typed word is large.

7. Japanese: kanji re-ranking and next phrase

For each Mozc conversion candidate c of the current reading, score log P(<ja> + committed text before the cursor + c) and re-rank. Combine with Mozc cost and user history rather than replacing them (see caveat in section 2). Example: かえる after 家に should prefer 帰る, and after 池の it should prefer 蛙.

Token healing is required. If the committed context ends inside a token, for example 「今日」 when training always saw 「今日は」 as one piece, the model is off-distribution and predicts badly. Fix: drop the last context token, then require the first predicted tokens to reproduce its text (mask the vocabulary to compatible pieces), and strip that text from the shown suggestion. This took next-phrase test cases from 2 of 9 to 6 of 9 hits. Reference: keyboard_eval.py LM.predict_continuations and _heal_mask_impl.

- **Next phrase (no active composition): run healed continuation search (beam about 8, up to 5 tokens) and show the top 3 strings.**
- **Kana input has no autocorrect, as the plan says.**
- **There is no U+2581 inside Japanese text: word-start tokens appear only at the start of a line or message.**

8. Settings and Stats UI

- Settings, Autocorrect: **Smart predictions (Zenith)** master switch, plus per-language toggles (en, es, ja).
- Stats: show the median and p95 latency per prediction and the model load time. Log locally whether a suggestion that was tapped came from Zenith's re-ordering; this gives you real keystroke-savings numbers to tune a, b and c.

9. Suggested order of work

1. Port the tokenizer to Kotlin. Unit-test it against tokenizer_kb.json outputs on 1,000 lines per language (they must match exactly).

2. Load zenith_kb.onnx with ONNX Runtime Mobile and run the single-token step. Measure latency and RAM on the Xiaomi 14 Civi before anything else; targets are ≤ 30 ms and ≤ 70 MB.

3. Golden test: for 20 fixed contexts, compare the top-5 tokens from Kotlin against the Python OnnxLM in keyboard_eval.py (the same file, so they should match exactly).

4. Add the cache plus incremental feeding plus cancellation. Then add English and Spanish candidate scoring and mixing.

5. Add Japanese re-ranking with token healing. Then add next-phrase.

6. Add Settings toggles and Stats. Dogfood it, and tune a, b, c.

7. Plan step 7: personal LoRA (the zenith finetune-personal command on the PC, from a MyKey backup) is not built yet. That needs work on the Zenith repo side; the Android side only needs to load an adapter later.

10. Retraining or re-exporting

```
# all from zenith/ with the project venv
python -m zenith.data.keyboard_corpus download|clean|pack     # data_kb/ (about 2.3 GB download)
python -m zenith.tokenizer.train_tokenizer --input data_kb/tok_sample/*.txt --output data_kb/tokenizer_kb.json
python -m zenith.training.train configs/zenith_keyboard.yaml   # about 20 h on the RTX 3050 laptop (thermal-thro
python -m zenith.export.export_onnx checkpoints_kb/latest.pt   # fp32 + SmoothQuant on the head
python -m zenith.export.quantize                               # -> export/zenith_kb.onnx (18.3 MB)
python -m zenith.export.export_tokenizer
python -m zenith.export.parity_test checkpoints_kb/latest.pt
python -m zenith.evaluation.keyboard_eval checkpoints_kb/latest.pt [--onnx export/zenith_kb.onnx]
```

Quantization details, in case you touch it: naive dynamic int8 destroyed the model (under 5% parity). The SwiGLU down-projections have wide activation ranges, so the transformer layers use int8 weight-only (DequantizeLinear plus fp32 MatMul). The 32k head is dynamic int8 after SmoothQuant (alpha 0.7), and the input embedding re-uses the head's int8 matrix with per-token scales (an extra Mul by 1/s undoes the smoothing). That is why the file is 18.3 MB rather than 28.5 MB.